



TECNOLÓGICO
NACIONAL DE MÉXICO



TEC DE
JUÁREZ
Forjando el futuro,
transformando vidas.

TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE CIUDAD JUÁREZ

DEPARTAMENTO DE SISTEMAS COMPUTACIONALES

NCIE

Digitalización de Procesos Internos del Nodo: Lista de Requerimientos para Despliegue v2.0

FECHA DE ENTREGA: 11 DE SEPTIEMBRE DE 2026

Gestor:

Edgar David Palacios Flores

Alumnos:

Oscar Chavez Muñoz (v1.0)

Joshua Velasco (v2.0)

Control de Versiones

Versión	Fecha	Autor	Descripción de Cambios
1.0	10 de Marzo, 2026	Oscar Chavez Muñoz	Emisión inicial del documento de despliegue para el portal del NCIE (servidor Ubuntu con Apache, PHP 8.1 y MySQL).
2.0	11 de Septiembre, 2026	Joshua Velasco	Actualización a la arquitectura v2.0 en contenedores Docker: requerimientos de hardware, software del anfitrión, imágenes y contenedores, PostgreSQL 16, red y puertos, seguridad y variables de entorno; consideraciones para Docker Desktop, migración de datos y despliegue sin contenedores.

Documento Confidencial | Propiedad de NCIE | Versión 2.0

Índice

1. Introducción	4
2. Objetivo	4
3. Alcance	4
4. Requerimientos de Despliegue	4
4.1. Requerimientos de Hardware	4
4.2. Requerimientos de Software del Anfitrión	5
4.3. Contenedores e Imágenes	5
4.4. Requerimientos de Base de Datos	5
4.5. Requerimientos de Red	6
4.6. Requerimientos de Seguridad	6
4.7. Variables de Entorno Mínimas	7
5. Consideraciones Adicionales	7

1. Introducción

El presente documento define los requerimientos técnicos necesarios para el despliegue de la **versión 2.0** del sistema del Nodo de Creatividad, Innovación y Emprendimiento (NCIE). El sistema está desarrollado con el framework PHP **Laravel 10** (10.48.29) y, a partir de esta versión, se distribuye y ejecuta como un conjunto de contenedores Docker orquestados con Docker Compose: **NGINX 1.27** como servidor web, **PHP-FPM 8.2** para la aplicación y **PostgreSQL 16** como motor de base de datos. Esta arquitectura corresponde a la "segunda versión adaptada a los estándares arquitectónicos de la institución" que anticipaba el documento 1.0 (base de datos PostgreSQL y servidor web estandarizado).

La misma definición de contenedores sirve para el entorno de desarrollo y demostración (equipo de escritorio con Docker Desktop) y para el servidor institucional (Linux con Docker Engine); las diferencias entre ambos se señalan expresamente.

2. Objetivo

Establecer de manera clara y verificable los requerimientos de hardware, software, base de datos, red y seguridad para el despliegue exitoso y seguro del portal NCIE v2.0, de modo que cualquier equipo que los cumpla pueda instalar el sistema siguiendo el Manual Técnico v2.0 (sección 5) y validar el resultado con la Bitácora de Pruebas v2.0.

3. Alcance

Este documento abarca los requerimientos técnicos de la versión 2.0 del portal NCIE (Laravel 10, PHP 8.2, PostgreSQL 16, NGINX, Docker Compose). Aplica al entorno local de desarrollo y a un servidor Linux con Docker designado por la institución. El despliegue tradicional sin contenedores (PHP y PostgreSQL instalados directamente en el sistema operativo) se contempla únicamente como alternativa en la sección 5; el motor MySQL de la versión 1.0 ya no forma parte de la arquitectura de referencia.

4. Requerimientos de Despliegue

4.1. Requerimientos de Hardware

Consumo verificado del stack en reposo con los datos de demostración: 59 MiB (aplicación), 21 MiB (PostgreSQL) y 8 MiB (NGINX) de memoria; 1.26 GB de imágenes en disco y 47 MB de datos. Los mínimos siguientes contemplan el sistema operativo, Docker y el crecimiento de datos, registros y respaldos.

Recurso	Mínimo	Recomendado	Observaciones
Procesador	2 vCPU	4 vCPU	x86_64 o ARM64 (verificado en Apple Silicon; las imágenes son multiarquitectura).
Memoria RAM	4 GB	8 GB	En macOS/Windows, Docker Desktop reserva memoria para su máquina virtual (4 GB por defecto); en equipos de 8 GB conviene limitarla a 3 GB. En Linux con Docker Engine bastan 4 GB para el stack.
Almacenamiento	20 GB SSD libres	40 GB SSD	Imágenes (~1.3 GB), código y dependencias (~250 MB), volumen de datos, fotos de proyectos, registros y respaldos locales (7 días).

Recurso	Mínimo	Recomendado	Observaciones
Red	100 Mbps	1 Gbps	Salida a internet durante la instalación (descarga de imágenes y dependencias).

4.2. Requerimientos de Software del Anfitrión

1. **Sistema operativo:** Ubuntu Server 22.04 LTS o superior (o Debian 12) para el servidor institucional; macOS 13 o superior, o Windows 10/11 con WSL 2, para desarrollo y demostración (verificado en macOS 26.6).
2. **Docker:** Docker Engine 24 o superior con el complemento **Docker Compose v2** (`docker compose`), o Docker Desktop 4.x en escritorio (verificado con Docker Desktop 4.85 / Engine 29.6.2). El usuario operador debe pertenecer al grupo `docker` en Linux.
3. **Git** para clonar el repositorio (`Joshua-Velasco/NCIEPortal`, GitHub) o un medio para copiar el árbol del proyecto.
4. **Proxy inverso con TLS** (solo producción): NGINX con Certbot o Caddy en el anfitrión, para publicar el sistema en 443 con certificado (Let's Encrypt).
5. **No se requieren en el anfitrión:** PHP, Composer, Node.js, npm ni PostgreSQL; todos se ejecutan dentro de los contenedores.

4.3. Contenedores e Imágenes

Definidos en `docker-compose.yml`, `docker/php/Dockerfile` y `docker/nginx/default.conf` del proyecto. Las imágenes se descargan de Docker Hub durante la construcción.

Servicio	Imagen	Versión verificada	Puerto	Función
nginx	nginx:1.27-alpine	1.27.5	80 → anfitrión 8080	Servidor web; sirve <code>public/</code> y reenvía PHP por FastCGI.
app	php:8.2-fpm-alpine (construida con <code>docker/php/Dockerfile</code>)	PHP 8.2.33, Composer 2.10.2	9000 (interno)	Aplicación Laravel; extensiones <code>pdo_pgsql</code> , <code>bcmath</code> , <code>zip</code> , <code>opcache</code> más las incluidas en la imagen (<code>mbstring</code> , <code>xml</code> , <code>curl</code> , <code>tokenizer</code> , <code>ctype</code> , <code>fileinfo</code> , <code>openssl</code> , <code>pdo</code> , <code>session</code>).
db	postgres:16-alpine	16.15	5432 → anfitrión 5432 (solo desarrollo)	Base de datos con volumen <code>pgdata</code> y <code>healthcheck</code> .
node (perfil assets)	node:20-alpine	20.x	—	Compilación opcional de assets con Vite; no se ejecuta en operación.

Los tres servicios permanentes usan `restart: unless-stopped`; `app` depende de que `db` esté *healthy*. El código del proyecto se monta en `/var/www/html` dentro de `app` y `nginx`, por lo que el directorio del proyecto debe residir en un disco local con permisos de escritura para `storage/`, `bootstrap/cache/` y `public/uploads/fotos/`.

4.4. Requerimientos de Base de Datos

- **SGBD:** PostgreSQL 16 (imagen `postgres:16-alpine`). Base de datos `ncie`, usuario `ncie` con privilegios completos sobre ella (crea tablas, secuencias y restricciones mediante las migraciones de Laravel).
- **Codificación y ordenamiento:** UTF8 con proveedor de ordenamiento ICU y locale `es-MX` (`POSTGRES_INITDB_ARGS`), para que las consultas ordenen acentos y ñ como en español. Verificado: `na < nz < ñu < oa`.

- **Esquema:** 20 migraciones (25 tablas) aplicadas con `php artisan migrate`; roles, permisos y usuario administrador con `--seed`; datos de demostración opcionales con `DemoSeeder`.
- **Persistencia:** volumen Docker `pgdata` montado en `/var/lib/postgresql/data`; sobrevive a `docker compose down` y se destruye solo con `down -v`.
- **Migración desde MySQL (v1.0):** opcional; crear el esquema con las migraciones y copiar únicamente los datos (`pgloader WITH data only, reset sequences` o volcados transformados), reiniciando las secuencias con `setval` y ejecutando `permission:cache-reset`.
- **RespalDOS:** `pg_dump -Fc` diario desde el contenedor y copia de `public/uploads/fotos`; retención de 7 días local más réplica externa (Manual Técnico, sección 10).

4.5. Requerimientos de Red

Acceso a internet: necesario durante la instalación para descargar imágenes (Docker Hub), dependencias PHP (Packagist) y, si se recompilan assets, paquetes npm. En operación, el servidor no necesita salida a internet, pero el navegador del usuario sí para las fuentes y bibliotecas servidas por CDN (`fonts.googleapis.com`, `cdn.jsdelivr.net`, `ui-avatars.com`).

Puerto	Protocolo / uso	Exposición requerida
8080/TCP (anfitrión)	HTTP hacia NGINX del contenedor (80). Acceso principal en desarrollo.	Local. En producción, solo accesible desde el proxy inverso del propio anfitrión (127.0.0.1).
80 y 443/TCP	HTTP/HTTPS del proxy inverso con certificado TLS.	Pública (firewall: permitir 80/443).
22/TCP	SSH para administración del servidor.	Restringida a la red de administración; acceso por clave.
5432/TCP (anfitrión)	PostgreSQL publicado por Compose para herramientas de escritorio.	Solo desarrollo. En producción no publicar o limitar a <code>127.0.0.1:5432</code> .
9000/TCP	FastCGI entre NGINX y PHP-FPM.	Interna a la red de Docker; no se publica.
5173/TCP	Servidor de desarrollo de Vite (<code>npm run dev</code>).	No se usa; el archivo <code>public/hot</code> no debe existir.

4.6. Requerimientos de Seguridad

- **Credenciales por defecto:** cambiar antes de exponer el sistema la contraseña del usuario administrador sembrado (`sistemancie@gmail.com / 12345678`) y la de PostgreSQL (`POSTGRES_PASSWORD` en `docker-compose.yml` y `DB_PASSWORD` en `.env`, por defecto `ncie_password`).
- **Archivo `.env`:** contiene claves y credenciales; no debe versionarse y debe tener permisos restrictivos (`chmod 600`). Debe generarse `APP_KEY` con `php artisan key:generate` tras la instalación.
- **Modo de producción:** `APP_ENV=production`, `APP_DEBUG=false`, `LOG_LEVEL=error`; `NCIE_EMAIL_VERIFICATION=true` con un servidor SMTP funcional (`MAIL_MAILER=smtp`).
- **Red:** HTTPS obligatorio en producción mediante el proxy inverso; puerto 5432 no publicado; firewall del anfitrión con 22, 80 y 443 únicamente.
- **Usuario del sistema:** administración con usuario estándar con `sudo`, acceso SSH por clave y `PermitRootLogin no`; el usuario operador en el grupo `docker`.

- **Protecciones incorporadas** (verificadas en la Bitácora v2.0): contraseñas con Bcrypt, tokens CSRF (419 sin token), límite de 5 intentos de acceso por minuto, permisos por rol en 101 rutas, bloqueo de `/.env` y `/.git` y ocultación de `X-Powered-By` en NGINX. Se recomienda añadir límite de tasa (`limit_req`) en el proxy si el sistema se expone a internet.
- **Actualizaciones y respaldos:** parches del sistema operativo y de Docker, renovación periódica de imágenes base y `composer audit`; respaldos diarios con prueba de restauración mensual.

4.7. Variables de Entorno Mínimas

Variable	Valor de desarrollo	Valor de producción
APP_ENV / APP_DEBUG	local / true	production / false
APP_URL	http://localhost:8080	https://<dominio-institucional>
APP_KEY	generada con <code>key:generate</code>	generada con <code>key:generate</code> (única por entorno)
DB_CONNECTION / DB_HOST / DB_PORT	pgsql / db / 5432	pgsql / db / 5432
DB_DATABASE / DB_USERNAME / DB_PASSWORD	ncie / ncie / ncie_password	ncie / ncie / contraseña fuerte
MAIL_MAILER (+ MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_ENCRYPTION, MAIL_FROM_ADDRESS, MAIL_FROM_NAME)	log	smtp con credenciales del servidor de correo institucional
NCIE_EMAIL_VERIFICATION	false	true
SESSION_DRIVER / CACHE_DRIVER / QUEUE_CONNECTION	file / file / sync	file / file / sync
LOG_CHANNEL / LOG_LEVEL	stack / debug	daily / error

5. Consideraciones Adicionales

Docker Desktop en equipos con poca memoria: en un Mac de 8 GB la máquina virtual de Docker toma por defecto 4 GB; con otras aplicaciones abiertas el arranque puede tardar o parecer que falla. Se recomienda fijar 3 GB en *Settings* → *Resources*, cerrar aplicaciones pesadas antes de abrir Docker y esperar 30–60 segundos a que `docker info` responda. Docker Desktop no tiene inicio automático, pero los contenedores del sistema se levantan solos cuando Docker arranca.

Entorno de desarrollo frente a producción: la configuración incluida en el proyecto (`.env.docker`) está pensada para desarrollo y demostración: modo debug activo, correo en modo log, verificación de correo desactivada, puerto 5432 publicado y contraseñas por defecto. Ninguno de esos valores debe llegar a producción; la sección 4.6 y el Manual Técnico v2.0 (secciones 5 y 11) enumeran los cambios obligatorios.

Datos de demostración: `php artisan db:seed --class=DemoSeeder` llena la base con áreas, gestores con horario, administrativos, alumnos, cursos, inscripciones, proyectos con foto, presupuestos y avisos (contraseña 12345678 en todas las cuentas). Es re-ejecutable y **borra** los datos existentes del dominio y los usuarios no administradores; no debe usarse sobre una base de producción.

Correo electrónico: el sistema envía correos de verificación de cuenta, restablecimiento de contraseña y buzón de sugerencias. La institución debe proporcionar un servidor SMTP con una cuenta remitente; las credenciales de Gmail usadas en 2025 quedaron inválidas. Mientras no exista SMTP, `MAIL_MAILER=log` evita errores y deja los correos en `storage/logs/laravel.log`. Las cuentas creadas desde el panel (administrativos, gestores, alumnos) requieren verificar su correo antes de usar el sistema; sin correo saliente, el administrador debe marcarlas como verificadas (pendiente P-08 del Manual Técnico).

Despliegue sin contenedores (alternativa): si el servidor institucional no admite Docker, los requerimientos equivalentes son PHP 8.2 con las extensiones BCMath, CType, cURL, DOM, Fileinfo, Filter, Hash, Mbstring, OpenSSL, PCRE, PDO, `pdo_pgsql`, Session, Tokenizer, XML y Zip; Composer 2; PostgreSQL 16 con base `ncie` en UTF8/ICU es-MX; NGINX con PHP-FPM (o Apache con `DocumentRoot` en `public/` y `mod_rewrite`); Node.js 20 solo para recompilar assets; y Git. `php artisan serve` sigue siendo exclusivamente para desarrollo y no debe usarse en producción.

Verificación del despliegue: tras instalar, confirmar `docker compose ps` (tres contenedores *Up*, base de datos *healthy*), `GET /` y `GET /login` con código 200, `GET /admin` redirigiendo a `/login`, `GET /.env` con 403, inicio de sesión del administrador y `php artisan test` en verde. La Bitácora de Pruebas v2.0 contiene la lista completa de casos de aceptación.